When a user sends a message, the system first checks if the user exists in the Upstash Redis database. If the user does not exist, a new entry is created with a message count of 1. If the user exists, the system increments the message count. If the message count exceeds the quota limit (e.g., 10), the message is rejected. Otherwise, the message is allowed, and the updated message count is stored in Upstash Redis.

Redis key format and management considerations

In this example, we used the format for the Redis key as follows:

```
const key = `message_count:${userId}:${today}`;
```

However, there are other ways to define this key format that can enhance data management. For instance, incorporating a date prefix can be beneficial as well:

```
const key = `${today}:message_count:${userId}`;
```

This structure allows for easier searching and targeted cleanup of old keys based on their date. For example, to delete all keys related to a specific day, you can filter by the date prefix. This allows you to clean up old keys and reduce the size of the database.

Next, notice that in the code we always increase the Redis count even if the count exceeds the message limit. We can improve this by implementing strategies to prevent abuse of the Redis store, especially when users can send multiple requests quickly:

- Implement rate limiting to restrict the number of requests a user can make within a defined timeframe so that the count does not get increased often. This means that we need to include the quota check after a rate limit middleware to ensure this feature is not abused.
- Introduce cooldown periods and enforce cooldowns after a user sends a message to limit immediate subsequent requests. This can prevent users from rapidly sending messages in succession.

Those additional security controls should safeguard your Redis store from abuse.

You can verify that this works by sending several messages to the AI application and then checking the Upstash dashboard. In the Data Browser section, you should see a Redis key with the format `message_count:{userId}:{YYYY-MM-DD}`. The corresponding value will be the number of messages sent by that user on that day. Once the message count reaches 10, any further interactions with the application on the same day will result in a 429 Too Many Requests status code. The UI can then display a message to the user, informing them that they have reached their message limit and can try again tomorrow. Figure 9.4 shows the message quote implementation.

Currently, the UI does not provide an indicator to inform users when they have exceeded their daily message limit. However, I encourage you to consider this as an exercise to further improve the application experience.